Koordynator projektu: Alicja Jędrzejewska
Programista (C++): Alicja Jędrzejewska, Łukasz Pniewski, Katarzyna Borysiak, Michał Urbańczyk
Programista backend (Python/Flask): Katarzyna Borysiak
Programista frontend (HTML/CSS/JS/Figma): Alicja Jędrzejewska, Katarzyna Borysiak
Tester: Łukasz Pniewski, Katarzyna Borysiak, Michał Urbańczyk
Autor dokumentacji: Alicja Jędrzejewska, Łukasz Pniewski, Katarzyna Borysiak, Michał Urbańczyk

Projekt:

"Optymalizacja transportu piwa w Shire: algorytmy przepływu i zarządzania kosztami naprawy dróg"

Opis ogólny:

Zadanie polega na znalezieniu maksymalnej ilości piwa, którą można dostarczyć do karczm w Shire przy ograniczeniach produkcji, transportu i pojemności browarów. W drugiej części problemu należy znaleźć rozwiązanie minimalizujące koszt naprawy dróg, przy zachowaniu maksymalnej ilości dostarczonego piwa. Na końcu trzeba wziąć pod uwagę zróżnicowanie wydajności pól jęczmienia w różnych ćwiartkach mapy oraz zaproponować system szybkiego wyszukiwania słów kluczowych.

Specyfikacja problemu

Dane wejściowe:

Program główny otwiera plik wejściowy lub pobiera dane od użytkownika. Wczytuje dane dotyczące ilości pól, browarów, karczm, skrzyżowań i krawędzi, a następnie pobiera dane związane z ćwiartkami - ich współrzędne oraz ilości rosnącego jęczmienia, oraz z polami, pobierając tylko ich współrzędne. Następnie pobiera dane odnośnie współrzędnych browarów i ilości produkowanego piwa, karczm i zapotrzebowania na piwo, oraz połączenia między obiektami z podaniem przepływu piwa, jęczmienia i kosztu naprawy drogi.

Dane wyjściowe:

Program zwraca maksymalny przepływ piwa do karczm i minimalny koszt naprawy dróg, w tym podaje w której ćwiartce znajduje się dane pole i ile na danym polu rośnie jęczmienia.

1.Problem pierwszy:

1.1 Dane wejściowe:

Osoba odpowiedzialna: Alicja Jędrzejewska

Do podanego wyżej problemu, stworzyłam serię klas; każda przechowuje określone informacje potrzebne do późniejszych obliczeń. Stworzonymi przeze mnie klasami były:

- Wezel - bazowa klasa dla wszystkich innych punktów
- Pole (dziedziczona po klasie Wezel) - ze zmienną *jeczmien* opisującą, ile jęczmienia może wyrosnąć na danym polu
- Browar (dziedziczona po klasie Wezel) - ze zmienną *przetworzenie* informującą, ile jęczmienia może przetworzyć dany browar
- Karczma (dziedziczona po klasie Wezel) - ze zmienną *zapotrzebowanie* informującą o tym, jakie jest zapotrzebowanie na piwo w danej karczmie
- Krawedz - opisująca krawędzie łączące punkty (dziedziczone po klasie Wezel), zawiera zmienne *start, koniec* - skąd/dokąd prowadzi droga, *przepustowosc_zboze* - ile zboża można przetransportować tą drogą, *przepustowosc_piwo* - ile piwa można przetransportować tą drogą, naprawa - do dalszych obliczeń związanych z inną częścią programu

Używając wyżej opisanych klas, zainicjowałam serię zmiennych, w których będą przechowywane dane w dalszej części programu. Najważniejsze z nich to macierz_sasiedztwa (potrzebna do obliczenia maksymalnego przepływu), a także wektory pola, browary, karczmy, krawędzie, które przechowują informacje zgodnie ze swoją nazwą.

1.2 Ford-Fulkerson:

Osoba odpowiedzialna: Alicja Jędrzejewska

Jednym z przedstawionych problemów było znalezienie maksymalnego przepływu w grafie. Do rozwiązania posłużył mi algorytm Forda-Fulkersona, który w pętli znajduje ścieżki powiększające w grafie. Można przez nie przesłać zboże czy piwo, a następnie zwiększać całkowity przepływ o minimalną wartość przepustowości na ścieżce.

Wewnątrz pętli while znajdujemy ścieżkę powiększającą, wyznaczamy minimalną przepustowość tej ścieżki, aktualizujemy macierz sąsiedztwa (w której przechowujemy dane), zwiększamy *maxprzeplyw*.

Najważniejsze segmenty:

- *grafkopia* - reprezentuje graf rezydualny,
- *rodzic[]* - dzięki tej tablicy możemy odtworzyć ścieżkę, przez którą właśnie przeszliśmy,
- *maxprzeplyw* - przechowuje łączny, aktualny maksymalny przepływ.

1.3 Edmonds-Karp:

Osoba odpowiedzialna: Alicja Jędrzejewska

Aby ukończyć implementacje algorytmu Forda-Fulkersona dodałam wyszukiwanie ścieżek powiększających, używając algorytmu BFS. Zmieniając kod na algorytm Edmondsa-Karpa i zapewniając tym samym złożoność O(VE^2), gdzie V to liczba wierzchołków w grafie, a E to liczba krawędzi.

Najważniejsze segmenty:

- *kolejka* - przechowuje wierzchołki grafu, które nie zostały jeszcze odwiedzone,
- *odwiedzone*[] - czy wierzchołek był już odwiedzony,
- *rodzic*[] - tak jak w algorytmie Forda-Fulkersona służy do odtworzenia ścieżki powiększającej,

graf[u][i] > 0 - sprawdza, czy istnieje krawędź, po której możemy się poruszać i czy wierzchołek został już odwiedzony.

1.4 Podsumowanie:

W każdej iteracji fordFulkerson() wywołuje algorytm bfs(), aby znaleźć ścieżkę od ujścia do źródła o najkrótszej liczbie krawędzi. Po jej znalezieniu następuje aktualizacja macierzy sąsiedztwa, a także maksymalnego przepływu.

2. Problem drugi: Zminimalizować koszt naprawy dróg używanych w transporcie zboża i piwa przy zachowaniu tej samej ilości piwa dostarczanego do karczm.

2.1 Modelowanie sieci transportowej.

Osoba odpowiedzialna: Łukasz Pniewski

Rozpatrywaną sieć transportową należy przedstawić jako graf skierowany. Wierzchołki reprezentują pola, browary, karczmy oraz skrzyżowania, natomiast krawędzie odpowiadają drogom, po których może poruszać się transport.

Ponadto, istnieją dwa dodatkowe wierzchołki: source, kontrolujący ilość produkowanego jęczmienia oraz target, kontrolujący zapotrzebowanie karczm na piwo. Każda krawędź ma przypisaną przepustowość (ile towaru można nią przewieźć) oraz koszt naprawy (czyli koszt transportu jednej jednostki towaru przez daną drogę). Krawędzie odwrotne posiadają koszty ujemne, aby w razie planowania tras, cofnąć koszty naprawy danej drogi. Dzięki temu struktura problemu przyjmuje postać sieci przepływowej z kosztami jednostkowymi. Graf nie może posiadać inicjalnie żadnych cykli ujemnych, tzn. na początku żadna krawędź odwrotna nie może posiadać dostępnej przepustowości. Trzeba również uwzględnić, że nie wszystkie drogi muszą być używane, czyli mogą zostać pominięte, jeśli nie są optymalne kosztowo.

2.2 Zachowanie ustalonej ilości dostarczonego piwa.

Osoba odpowiedzialna: Łukasz Pniewski

Problem ten opisuje konieczność utrzymania dotychczasowej ilości piwa dostarczanego do karczm. Oznacza to, że przepływ w sieci musi mieć określoną wartość, wyznaczoną wcześniej jako największy możliwy do osiągnięcia. Algorytm wyznaczający minimalny koszt tego przepływu nie może zmniejszyć tej wartości, nawet jeśli pozwoliłoby to obniżyć koszty. To sprawia, że nie wystarczy tylko znaleźć tanią konfigurację przepływu, ale musi ona również realizować pełne zapotrzebowanie. Szukamy więc najtańszego planu przepływu o zadanej wartości. Technicznie można to zrealizować np. przez algorytm Successive Shortest Path, przy narzuconej wymaganej wartości przepływu.

2.3 Znajdowanie najtańszych ścieżek.

Osoba odpowiedzialna: Łukasz Pniewski

Jednym z kluczowych elementów rozwiązania problemu jest znajdowanie ścieżek o najniższym koszcie. W kontekście przepływu w sieci oznacza to konieczność wielokrotnego wyszukiwania najtańszych ścieżek od źródła do ujścia, które pozwolą przesłać kolejne porcje towaru przy jak najmniejszym wydatku. Standardowym podejściem w takich przypadkach jest zastosowanie algorytmu Dijkstry, który wyznacza najkrótsze ścieżki w grafach o nieujemnych wagach. Do radzenia sobie z wagami ujemnymi, dołącza się “potencjały” wierzchołków, czyli zapisywane najtańsze koszty dojścia do danego wierzchołka. Proces ten powtarzany jest aż do momentu, gdy wymagana ilość piwa zostanie dostarczona do karczm.

2.4 Obsługa ujemnych kosztów.

Osoba odpowiedzialna: Łukasz Pniewski

Ze względu na strukturę grafu i obecność tzw. krawędzi rezydualnych, w algorytmie mogą pojawiać się ujemne koszty przejścia między wierzchołkami. To utrudnia zastosowanie klasycznych algorytmów najkrótszych ścieżek, takich jak Dijkstra, które wymagają nieujemnych wag. Rozwiązaniem tego problemu jest wprowadzenie tzw. **potencjałów wierzchołków** – specjalnych wartości przypisywanych każdemu wierzchołkowi, na podstawie których przekształca się koszty wszystkich krawędzi w nowe, nieujemne wartości. Dzięki temu przekształceniu, nadal można stosować szybki algorytm Dijkstry, a wyznaczone ścieżki zachowują swoją poprawność. Potencjały są aktualizowane po każdej iteracji przepływu, co pozwala na bezpieczne działanie algorytmu Dijkstry w każdej nowej iteracji i uniknięcie cykli ujemnych. Ten mechanizm jest kluczowy w algorytmie Successive Shortest Path i pozwala na optymalizację kosztową przy zachowaniu wysokiej wydajności działania.

3.Problem trzeci:

“Okazało się jednak, że w różnych ćwiartkach Shire ilości jęczmienia wyrastające na jednym polu różnią się od siebie. W tym celu Samwise kazał zebrać współrzędne punktów granicznych każdej ćwiartki (każda z ćwiartek okazała się być wielokątem wypukłym, rozłącznym z pozostałymi ćwiartkami). Wie też, ile jęczmienia wyrasta na polu w poszczególnych ćwiartkach. Zmodyfikujcie swoje rozwiązanie uwzględniając te nowe informacje zebrane przez burmistrza Sama.”

3.1 Dodanie nowych klas:

Osoba odpowiedzialna: Katarzyna Borysiak

W tej części programu dodano klasę Cwiartka, aby reprezentowała powierzchnie jednej z czterech ćwiartek oraz uzupełniono klasę Pole oraz Wezel, by uwzględniały tą część zadania.

Klasa Wezel reprezentuje punkt w przestrzeni dwuwymiarowej, została do niej dodana implementacja algorytmu Grahama do wyznaczania otoczki wypukłej, pomocnicza metoda do obliczania iloczynu wektorowego oraz metoda do obliczania pola powierzchni na podstawie otoczki.

Klasa Cwiartka stanowi reprezentacje jednej z 4 ćwiartek terenu. Umożliwia dodawanie wierzchołków definiujące granice tej ćwiartki, przypisywanie jej

wydajności, a także obliczanie pola powierzchni obszaru, który opisuje. Do wyznaczenia pola powierzchni wykorzystywana jest otoczka wypukła, obliczana przy użyciu algorytmu Grahama. Ponadto, klasa implementuje algorytm liczby nawinięć, który służy do określenia, czy dany punkt znajduje się wewnątrz wielokąta utworzonego z granicznych punktów ćwiartki lub bezpośrednio na jego krawędzi.

W klasie Pole została zmieniona interpretacja pola z punktu jako zbiór punktów jednego pola oraz usunięta możliwość ręcznego przypisywania ilości jęczmienia, ilość ta jest obliczana za pomocą funkcji obliczCalkowitaWartoscJeczmienia(). Klasa Pole umożliwia dodawanie wierzchołków definiujące granice pola i oblicza pole powierzchni z użyciem algorytmu Grahama. Oblicza też wartość jęczmienia przypadającą na dane pole na podstawie powierzchni ćwiartki i jej wydajności. Dla każdego punktu tworzącego granicę pola sprawdzane jest, do której ćwiartki należy. W przypadku, gdy pole przypisuje się do więcej niż jednej ćwiartki lub wartość jęczmienia jest mniejsza niż jeden, generowany jest odpowiedni komunikat o błędzie. Wyznacza środek geometryczny pola by pole było uznawane jako punkt w algorytmie Edmondsa-Karpa.

3.2 Algorytm Grahama:

Osoba odpowiedzialna: Katarzyna Borysiak

“W tym celu Samwise kazał zebrać współrzędne punktów granicznych każdej ćwiartki (każda z ćwiartek okazała się być wielokątem wypukłym, rozłącznym z pozostałymi ćwiartkami).”

Użyto w tym celu algorytmu Grahama, który służy do wyznaczania otoczki wypukłej (ang. *convex hull*) zbioru punktów na płaszczyźnie. Otoczka wypukła to najmniejszy wielokąt wypukły, który zawiera wszystkie punkty zbioru. W implementacji zastosowano klasyczne podejście algorytmu Grahama, które składa się z kilku głównych etapów:

```
Wezel::Wezel(int x_, int y_) : x(x_), y(y_) {}

bool Wezel::operator<(const Wezel& p) const {
    return(y < p.y) || (y == p.y && x < p.x);
}
```

Operator < definiuje sposób porównywania punktów. Najpierw porównywana jest współrzędna y, punkt z najmniejszą wartością jest traktowany jako "mniejszy". W przypadku remisu decyduje współrzędna x. Jest to kluczowe przy wyborze punktu startowego do algorytmu Grahama.

```
int Wezel::iloczynWektorowy(const Wezel& O, const Wezel& A, const Wezel& B) {
    return (A.getX() - O.getX()) * (B.getY() - O.getY()) -
        (A.getY() - O.getY()) * (B.getX() - O.getX());
}
```

Ta funkcja oblicza iloczyn wektorowy dwóch wektorów: **OA** i **OB**. Wynik dodatni oznacza skręt w lewo, a ujemny skręt w prawo, zero oznacza współliniowość punktów. Iloczyn wektorowy jest używany do sprawdzania, czy aktualnie przetwarzane punkty tworzą "zakręt w lewo" i powinny zostać dodane do otoczki, czy też trzeba cofnąć się i usunąć ostatni punkt.

```
vector<Wezel> Wezel::algorytmGrahama(const vector<Wezel>& wezly) {
    if (wezly.size() <= 3) return wezly;
```

Główna funkcja algorytmu Grahama działa w następujących krokach:

Najpierw sprawdza ilość punktów, jeśli zbiór zawiera 3 lub mniej punktów, są one zwracane jako wynik (są już otoczką). Następnie wybierany jest punkt początkowy o najmniejszej współrzędnej y, a w razie remisu o mniejszym x. Potem są sortowane punkty za pomocą zewnętrznej funkcji MergeSort()

```
    vector<Wezel> posortowaneWezly = wezly; // kopia do sortowania
    mergeSort(posortowaneWezly, 0, posortowaneWezly.size() - 1, start);
```

Na koniec budowana jest otoczka na podstawie posortowanych punktów. Iteracyjnie tworzona jest lista punktów otoczki, dla każdego nowego punktu sprawdzana jest orientacja trzech ostatnich punktów. Jeżeli nie tworzą skrętu w lewo, ostatni punkt jest usuwany. Funkcja zwraca wektor punktów tworzących otoczkę wypukłą, ułożoną w porządku przeciwnym do ruchu wskazówek zegara.

```
vector<Wezel> powloka;
for (const Wezel& p : posortowaneWezly) {
    while (powloka.size() >= 2 && iloczynWektorowy(powloka[powloka.size() - 2], powloka[powloka.size() - 1], p) <= 0) {
        powloka.pop_back();
    }
    powloka.push_back(p);
}
```

Aby poprawnie zastosować algorytm Grahama do znajdowania otoczki wypukłej, punkty muszą zostać posortowane według kąta, jaki tworzą z wybranym punktem

początkowym. W tym celu użyto algorytmu sortowania przez scalanie, którego głównymi elementami implementacji jest:

```
Wezel start(0,0);
```

Użycie zmiennej globalnej start, która przechowuje punkt odniesienia ten sam, który został wybrany jako punkt początkowy w algorytmie Grahama. Jest używany w funkcji porównującej punkty pod kątem względem tej zmiennej.

```
bool porownanieSkrajnychPunktow(const Wezel& a, const Wezel& b) {
```

Następnie funkcja porównuje dwa punkty a i b względem punktu odniesienia start. Działa według następujących zasad:

Iloczyn wektorowy określa, który punkt znajduje się “najbardziej w lewo” względem drugiego. Większy iloczyn oznacza większy kąt, a więc wyższy priorytet.

Jeśli punkty są współliniowe względem start, bliższy punkt jest traktowany jako “mniejszy”.

Funkcja zapewnia poprawne sortowanie według kąta polarnego.

```
}
void merge(vector<Wezel>& punkty, int lewy, int srodek, int prawy) {
    vector<Wezel> L(punkty.begin() + lewy, punkty.begin() + srodek + 1);
```

Funkcja merge scala dwie posortowane części wektora punkty w jeden posortowany ciąg. Wykorzystuje pomocnicze wektory L i R jako bufor dla lewej i prawej połowy. Porównanie elementów odbywa się za pomocą funkcji porownanieSkrajnychPunktow().

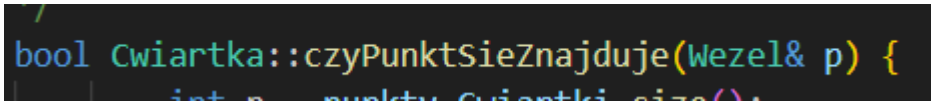
```
void mergeSort(vector<Wezel>& punkty, int lewy, int prawy, Wezel ref) {
    start = ref;
```

Rekurencyjna funkcja dzieląca dane na mniejsze podzbiory aż do pojedynczych elementów, a następnie scala je za pomocą merge. Zmienna ref przekazywana do funkcji ustawia punkt start, który służy jako odniesienie przy sortowaniu punktów.

3.3 Algorytm Winding-number:

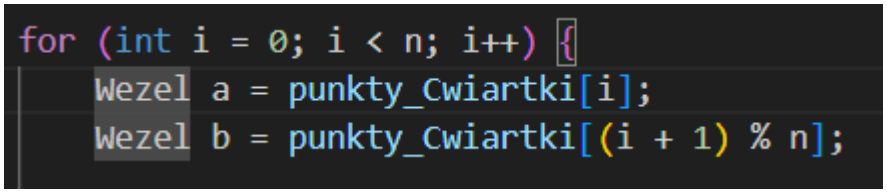
Osoba odpowiedzialna: Katarzyna Borysiak

Założono, że Samwise ma punkty graniczne każdej ćwiartki i pola, także każde pole może być tylko w jednej ćwiartce. Algorytm Winding Number (liczba nawinięć) służy do sprawdzania, czy dany punkt znajduje się wewnątrz wielokąta, w tym przypadku czy dane pole znajduje się w jednej z czterech ćwiartek. W odróżnieniu od prostszych metod (np. algorytmu parzysto-nieparzysto, ang. *even-odd*), Winding Number bierze pod uwagę kierunek przechodzenia po krawędziach wielokąta. Działa niezawodnie zarówno dla wielokątów wypukłych, jak i wklęsłych.

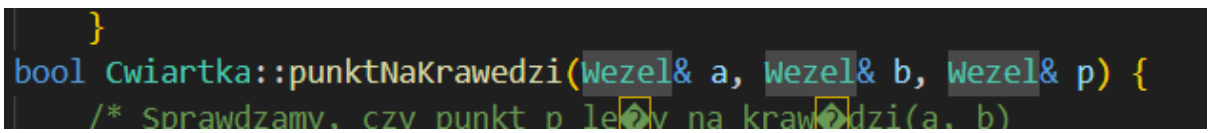


Sprawdza czy punkt p znajduje się wewnątrz wielokąta zdefiniowanego przez punkty_Cwiartki. Główne etapy funkcji:

Tworzona jest zmienna windingNumber, która będzie liczyć liczbę nawinięć krawędzi wielokąta wokół punktu p. Dla każdej krawędzi (a, b) wielokąta:



Użycie (i+1) %n zapewnia cykliczne przejście od pierwszego punktu do ostatniego. Jeśli punkt znajduje się na dowolnej krawędzi, zwracana jest wartość true, oznacza to, że punkt należy do wielokąta. W zależności od kierunku przechodzenia przez punkt p, windingNumber jest inkrementowany lub dekrementowany. Jeśli krawędź przechodzi z dołu do góry i punkt leży po lewej stronie tej krawędzi to zwiększamy licznik. Jeśli przechodzi z góry do dołu i punkt leży po prawej stronie to zmniejszamy licznik. Po przetworzeniu wszystkich krawędzi sprawdzany jest wynik, jeśli liczba nawinięć jest różna od zera, oznacza to, że punkt leży wewnątrz wielokąta, w przeciwnym razie na zewnątrz.
Algorytm wykorzystuje także funkcję pomocniczą



Sprawdza, czy punkt p leży dokładnie na odcinku łączącym punkty a i b. Weryfikacja odbywa się dwuetapowo, najpierw sprawdzana jest współliniowość,

obliczany jest iloczyn wektorowy. Jeżeli jest różny od zera (z tolerancją 1e-9), punkt nie leży na tej samej linii. Sprawdzany jest także zakres współrzędnych, czy punkt p mieści się w zakresie współrzędnych x i y odcinka a i b.

3.4 Wyliczenie ilości jęczmienia dla każdego pola:

Osoba odpowiedzialna: Katarzyna Borysiak

"Okazało się jednak, że w różnych ćwiartkach Shire ilości jęczmienia wyrastające na jednym polu różnią się od siebie"

Założono, że każda z ćwiartek ma swoją wydajność i wpływa ona na ilość rosnącego jęczmienia. Ustalono też, że ilość jęczmienia rośnie proporcjonalnie do powierzchni. Na tej podstawie wykorzystując wzór na proporcje Samwise może sam obliczyć, ile wyrasta jęczmienia na każdym polu. Najpierw w klasie Pole pobierana jest dla danej ćwiartki jej wydajność.

```
double Pole::obliczCalkowitaWartoscJeczmienia(Cwiartka& ktora)
{
    double Jcale = ktora.zwrocWydajnosc();
    double Pcale = ktora.onliczPoleCwiartki();
    double Ppole = this->obliczPolePola();
    double wynik = (Jcale / Pcale) * Ppole;
    return wynik;
}
```

Aby oszacować ilość jęczmienia możliwą do zasiania na danym polu, konieczne jest obliczenie jego powierzchni. Zakładając, że granice pola są reprezentowane jako wielokąt wypukły, powierzchnia ta może być łatwo wyznaczona przez podział na trójkąty i zsumowanie ich pól.

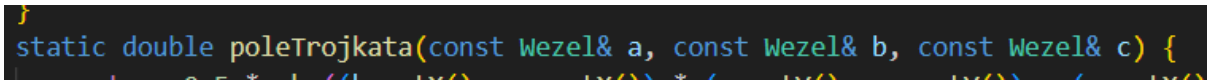
```
}
double Wezel::obliczPole(const vector<Wezel>& powloka) {
```

Funkcja oblicza pole wielokąta opisanego przez punkty w wektorze powloka. Zakłada, że punkty są uporządkowane przeciwnie do ruchu wskazówek zegara i tworzą wypukły wielokąt. Najpierw inicjalizuje zerową wartość pole, następnie funkcja iteruje po punktach i tworzy trójkąty z pierwszym punktem oraz kolejnymi parami punktów za pomocą funkcji pomocniczej poleTrojkata().

```
for (size_t i = 1; i < powloka.size() - 1; i++) {
    pole += poleTrojkata(powloka[0], powloka[i], powloka[i + 1]);
}
```

Jest to tzw. triangulacja wachlarzowa, która pozwala łatwo i szybko zsumować

pole całego wielokąta. Na koniec zwracana jest wartość pola.



Funkcja pomocnicza obliczająca pole trójkąta utworzonego przez trzy punkty a, b i c. Wykorzystuje wzór na pole trójkąta z iloczynu wektorowego:

$$\text{Pole} = \frac{1}{2} \cdot |(b_x - a_x)(c_y - a_y) - (c_x - a_x)(b_y - a_y)|$$

Ten wzór zapewnia dokładny wynik bez potrzeby obliczenia długości boków trójkąta.

Po wyznaczeniu powierzchni danej ćwiartki i pola można obliczyć, ile jęczmienia ono wytwarza. Korzystając ze wzoru na proporcje zwraca szacowaną ilość jęczmienia przypadająca na to konkretne pole.

$$\text{jęczmień na pole} = \left(\frac{J_{\text{ćw}}}{P_{\text{ćw}}} \right) \cdot P_{\text{pole}}$$

4.Problem czwarty:

“Burmistrz chce również wykorzystać skromne zasoby pamięci komputera, żeby zapisać sobie Wasze rozwiązania na przyszłość. Marzy o tym, żeby szybko wyszukiwać w tych rozwiązaniach słów: „piwo”, „jęczmień”, „browar” oraz innych, które przyjdą mu kiedyś do głowy. Chciałby przetestować kilka sposobów wyszukiwania słów.”

4.1 Zapis danych wyjściowych:

Osoba odpowiedzialna: Michał Urbańczyk

“Burmistrz chce również wykorzystać skromne zasoby pamięci komputera, żeby zapisać sobie Wasze rozwiązania na przyszłość.” Zdecydowaliśmy się więc zapisywać dane w formacie .bin, wykorzystując kodowanie Huffmana, co pozwala na kompresję danych w postaci binarnej. Struktura pliku binarnego wygląda następująco:

- Nagłówek: zawiera liczbę różnych znaków w tekście oraz uporządkowaną mapę (znak, liczba wystąpień).

- Zakodowana długość tekstu.
- Zakodowany tekst.

Działanie algorytmu kompresji:

1. Sprawdzamy przypadek skrajny (czy tekst jest pusty).
2. Tworzymy uporządkowaną mapę <znak, liczba wystąpień>.
3. Budujemy drzewo kodowe zgodne z algorytmem Huffmana.
4. Na podstawie drzewa przypisujemy każdemu znakowi odpowiedni kod binarny.
5. Zamieniamy cały tekst na ciąg 0 i 1, dzielimy go na bloki po 8 bitów i zapisujemy jako bajty (char) do pliku.

4.2 Odczyt zakodowanych danych:

Osoba odpowiedzialna: Michał Urbańczyk

Skoro zakodowaliśmy nasze danę musimy je umieć odczytać. Musi więc wiedzieć jak zbudowane było drzewo wykorzystane do zakodowania. Odczytujemy więc nagłówek naszego pliku, który nie jest zakodowany, żeby pozyskać mapę <znak, ilość wystąpień znaku>, dzięki której jesteśmy wstanie zbudować identyczne drzewo zgodne z algorytmem Huffman-a. Teraz wystarczy przejść przez nasz binarny tekst po tym drzewie, żeby otrzymać odkodowane znaki ostatecznie tworząc identyczny do początkowego (przed kompresją) tekstu.

4.3 Algorytm Knutha-Morrisa-Pratta:

Osoba odpowiedzialna: Michał Urbańczyk

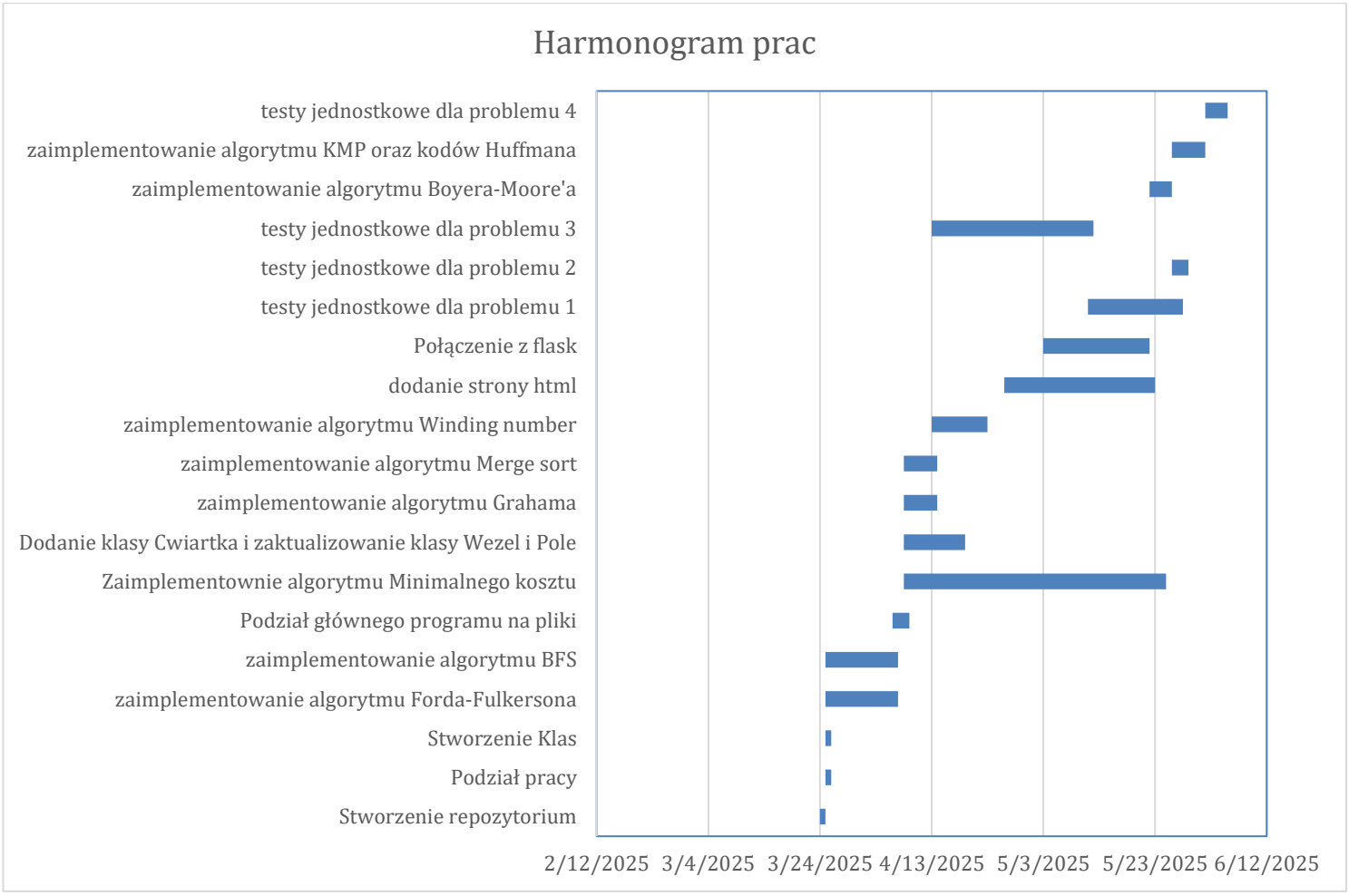
“Marzy o tym, żeby szybko wyszukiwać w tych rozwiązaniach słów: „piwo”, „jęczmień”, „browar” oraz innych, które przyjdą mu kiedyś do głowy. Chciałby przetestować kilka sposobów wyszukiwania słów.” Musimy w takim razie zrobić wyszukiwarkę do naszego pliku wynikowego. Najpierw odczytujemy tekst naszego pliku za pomocą dekodera. Mając już tekst w normalnej formie przechodzimy do algorytmu Knutha-Morrisa-Pratta omawianego na wykładzie. Budujemy więc tablicę prefikso-sufiksów zgodnie z pseudokodem z wykładu. Mając już tą tablice możemy przejść do wyszukiwania zgodnie z algorytmem Knutha-Morrisa-Pratta. Działanie to zwraca nam tablicę wystąpień wzorca w tekście, którą wykorzystujemy do odpowiedniego wypisania danych.

4.3 Algorytm Boyera-Moore’a:

Osoba odpowiedzialna: Michał Urbańczyk

Burmistrz chciał mieć kilka sposobów wyszukiwania, więc dodajemy wyszukiwanie za pomocą algorytmu Boyera-Moore'a, żeby miał swobodę wyboru. Zgodnie z algorytmem tworzymy dwie pomocnicze tablicę (zgodne z heurystykami: bad-character-heuristic oraz good-suffix-heuristic). Mając je wyznaczone możemy przejść do wyszukiwania algorytmem Boyera-Moore'a, żeby uzyskać tablicę wystąpień wzorca w tekście, którą wykorzystujemy do odpowiedniego wypisania danych. (Wszystko zgodnie z pseudokodem z wykładu, tylko zamienione tak żeby tablicę były liczone od 0 nie od 1)

Harmonogram realizacji Projektu



Ogólny opis przebiegu prac:

Prace nad projektem rozpoczęły się 24 marca 2025 roku od założenia repozytorium, co umożliwiło efektywne zarządzanie kodem źródłowym i współpracę zespołową. Następnego

dnia odbył się podział obowiązków oraz rozpoczęto implementację podstawowych komponentów projektu, w tym klas oraz algorytmów kluczowych dla działania programu.

W kolejnych dniach wdrożono algorytmy Forda-Fulkersona i BFS, które były rozwijane równolegle w okresie od 25 marca do 6 kwietnia. W międzyczasie kontynuowano rozbudowę struktury projektu – m.in. 6 kwietnia dokonano podziału głównego programu na pliki, co poprawiło przejrzystość i modularność kodu.

Od 8 kwietnia rozpoczęto implementację kolejnych algorytmów: Minimalnego Kosztu (rozwijanego do dnia 1 czerwca), Grahama, Merge Sort oraz Winding Number. W międzyczasie w dniu 13 kwietnia dodano klasę Cwiartka oraz zaktualizowano klasy Wezel i Pole (z uporządkowaniem ich 18 kwietnia), co stanowiło zaimplementowanie większości wymaganych algorytmów.

Na przełomie kwietnia i maja zespół rozpoczął prace nad interfejsem użytkownika – dodano stronę HTML oraz zintegrowano projekt z frameworkiem Flask, co pozwoliło na stworzenie działającej aplikacji webowej.

W maju rozpoczęto etap testowania. Testy jednostkowe były realizowane w podziale na cztery główne problemy i trwały aż do końca projektu – od 13 kwietnia do 1 czerwca. Równolegle, w dniach od 22 maja do 4 czerwca, zaimplementowano kluczowe algorytmy: Boyera-Moore'a, Knutha-Morrisa-Pratta i Huffmana oraz odpowiednie testy co stanowiło domknięcie funkcjonalności aplikacji. W między czasie została dokończona strona i połączenia flask z częścią programu wyszukiwarki. Ostatnim etapem było dokończenie pisania Dokumentacji.

Zastosowane algorytmy

W ramach realizacji projektu zaimplementowano kilka kluczowych algorytmów geometrycznych i numerycznych, które umożliwiają efektywne przetwarzanie i analizę danych przestrzennych. Poniżej opisano najważniejsze z nich:

Algorytm Edmondsa-Karpa jest jedną z realizacji algorytmu Forda-Fulkersona z użyciem BFS, która jest stosowana przy rozwiązywaniu problemu maksymalnego przepływu w grafie. Działa w czasie O(VE^2). Algorytm ten jest często uznawany za bardzo użyteczny szczególnie dla grafów rzadkich. W uproszczeniu działa w następujący sposób: szukamy ścieżki powiększającej o najmniejszej liczbie krawędzi, sprawdzamy jej przepływ, dodajemy go do maksymalnego przepływu i "usuwamy" daną ścieżkę tak, aby po zakończeniu obliczeń nie została nam żadna ścieżka powiększająca, którą możemy podążyć.

Algorytm Successive Shortest Path znajduje minimalny koszt maksymalnego przepływu w grafie, który został znaleziony dzięki algorytmowi Edmondsa-Karpa. Za każdym razem wyszukuje najtańszą ścieżkę za pomocą algorytmu Dijkstry, następnie w każdej iteracji określana jest maksymalna możliwa ilość przepływu, którą da się przesłać przez nowo znalezioną ścieżkę. Po tym, algorytm modyfikuje przepustowości krawędzi oraz dodaje koszt naprawy, proporcjonalny do liczby jednostek przesłanych przez każdą drogę. Wykorzystanie potencjałów umożliwia uniknięcie problemu ujemnych kosztów i gwarantuje poprawność działania Dijkstry. Proces powtarzany jest do momentu, aż zostanie osiągnięty wymagany przepływ lub nie da się już znaleźć ścieżki. Zwraca parę: rzeczywisty przepływ i odpowiadający mu minimalny koszt naprawy dróg. Działa on w czasie $O(F * V^2)$ (gdzie F to maksymalny przepływ, a V to ilość wierzchołków w grafie).

Algorytm Dijkstry znajduje najkrótszą ścieżkę (o minimalnym koszcie) w rozważanym grafie, uwzględniając tzw. potencjały wierzchołków. Jest to zmodyfikowana wersja klasycznego algorytmu Dijkstry, która działa poprawnie nawet przy ujemnych kosztach, dzięki przekształceniu ich na nieujemne. Do śledzenia najkrótszych ścieżek wykorzystuje kolejkę priorytetową, która wybiera zawsze wierzchołek o najmniejszym koszcie dojścia, a jeśli są identyczne – o najmniejszym indeksie. Każdorazowo aktualizuje koszt dojścia do wierzchołka oraz jego rodzica, jeśli uda się znaleźć korzystniejszą drogę. Gdy dotrze do wierzchołka docelowego `t`, zapamiętuje, że znaleziono ścieżkę, ale kontynuuje przeszukiwanie, by znaleźć potencjalnie lepsze drogi. Potencjały odgrywają kluczową rolę — pozwalają przekształcić wagi w tzw. efektywne koszty, zapewniając brak ujemnych wartości, a tym samym brak zapętlania się algorytmu. Funkcja kończy się, gdy kolejka jest pusta, i zwraca informację, czy istnieje ścieżka z `s` do `t`. Wektor *rodzic* zawiera zakodowaną ścieżkę, którą można później prześledzić w algorytmie SSP. Działa w czasie $O(V^2)$ (gdzie V to ilość wierzchołków grafu), ze względu na wykorzystanie macierzy sąsiedztwa.

Algorytm Grahama (ang. *Graham scan*) Działa w czasie O (n log n). Został wykorzystany do obliczenia otoczki wypukłej zbioru punktów, czyli najmniejszego wypukłego wielokąta obejmującego wszystkie wierzchołki danego obszaru. Algorytm działa w dwóch krokach: najpierw wybierany jest punkt o najmniejszej współrzędnej y (w przypadku remisu – o najmniejszej współrzędnej x), a następnie pozostałe punkty są sortowane względem kąta nachylenia względem tego punktu odniesienia. Do sortowania punktów zaimplementowano zmodyfikowaną wersję algorytmu sortowania przez scalanie (*merge sort*), która uwzględnia kolejność kątową względem punktu startowego. Następnie konstruowana jest otoczka poprzez iteracyjne dodawanie punktów, przy jednoczesnym sprawdzaniu skrętów (lewy, prawy, współliniowy), co gwarantuje zachowanie wypukłości wielokąta.

Do posortowania punktów według kąta względem punktu odniesienia (startowego) w algorytmie Grahama wykorzystano zmodyfikowany algorytm sortowania przez scalanie **MergeSort**, którego złożoność czasowa wynosi O (n log n). Klasyczny algorytm merge sort

jest algorytmem typu „dziel i zwyciężaj”, który dzieli dane wejściowe na mniejsze części, sortuje je rekurencyjnie, a następnie scala w jedną, uporządkowaną całość. W kontekście algorytmu Grahama konieczne było zastąpienie klasycznego porównania liczb funkcją porównującą kąty punktów względem punktu startowego. Zrealizowano to poprzez porównanie wartości iloczynu wektorowego, a w przypadku współliniowości – długości wektora względem punktu odniesienia.

Do określenia, czy dany punkt znajduje się wewnątrz wielokąta wykorzystano **algorytm Winding number**, który działa w czasie O(n). Jest to algorytm geometrii obliczeniowej, który analizuje sposób, w jaki granice wielokąta „owijają się” wokół badanego punktu. W szczególności, dla każdej krawędzi sprawdzane jest czy zachodzi zmiana orientacji wektora względem punktu, innymi słowy, czy następuje obrót wokół tego punktu. Jeśli suma zmian orientacji (tzw. liczba nawinięć) jest równa zero, oznacza to, że punkt leży na zewnątrz obszaru. W przeciwnym przypadku uznaje się, że punkt znajduje się wewnątrz wielokąta. Dodatkowo, w algorytmie uwzględniono przypadek, w którym punkt znajduje się dokładnie na jednej z krawędzi, wtedy również zostaje zaliczony do wnętrza.

Obliczanie Pola Powierzchni wielokąta działa w czasie O(n). Pole powierzchni wielokąta jest obliczane na podstawie otoczki wypukłej, przy użyciu dekompozycji na trójkąty. Metoda ta polega na wybraniu jednego wierzchołka jako punktu odniesienia i obliczaniu sumy pól kolejnych trójkątów utworzonych z tym punktem oraz dwóch sąsiednich punktów otoczki. Do obliczenia pola każdego trójkąta wykorzystuje się wzór oparty na obliczeniu iloczynu wektorowego macierzy, co zapewnia dużą dokładność oraz zgodność z zasadami geometrii analitycznej.

Do zapisu wyników stosujemy:

Algorytm Huffmana to klasyczny algorytm bezstratnej kompresji danych, który umożliwia efektywne kodowanie tekstu przy minimalnym zużyciu pamięci. Złożoność czasowa tego algorytmu wynosi O(n log n), gdzie *n* to liczba unikalnych znaków w tekście. Algorytm działa w kilku krokach:

1. Zliczanie częstości występowania każdego znaku w tekście.
2. Budowanie drzewa Huffmana na podstawie uporządkowanej (rosnąco) mapy <znak, liczba wystąpień>, przy użyciu kolejki priorytetowej.
3. Przypisanie każdemu znakowi binarnego kodu (krótszy kod dla częstszych znaków).
4. Zamiana tekstu na ciąg bitów na podstawie wyznaczonych kodów oraz zapis do pliku w postaci binarnej, z dołączonym nagłówkiem (zawierającym mapę częstości znaków i długość pierwotnego tekstu).

Algorytm zapewnia znaczne zmniejszenie rozmiaru danych przy dużych plikach zachowaniu ich pełnej rekonstrukcji, co jest kluczowe przy ograniczonych zasobach pamięciowych.

Do wyszukiwania wzorca wykorzystujemy dwa algorytmy:

Algorytm Knutha-Morrisa-Pratta to klasyczny algorytm wyszukiwania wzorca w tekście o złożoności O(n + m), gdzie *n* to długość tekstu, a *m* – długość wzorca.

Zasada działania:

1. Budowa tzw. tablicy prefikso-sufiksów, która określa najdłuższy właściwy prefiks wzorca będący jednocześnie jego sufiksem.
2. Przeszukiwanie tekstu bez cofania indeksu w tekście, co znacząco przyspiesza działanie wyszukiwarki.

Dzięki zastosowaniu KMP, wyszukiwanie słów takich jak „piwo", „browar" czy „jęczmień" w dużych plikach tekstowych odczytanych z zakodowanych danych może być wykonane w czasie liniowym względem długości tekstu. Drugim algorytmem do wyszukiwania jest:

Algorytm Boyera-Moore'a to jeden z algorytmów wyszukiwania tekstu, który świetnie sprawdza się przy długich wzorcach i dużych tekstach. Jego złożoność w pesymistycznym przypadku wynosi O(n * m), ale w praktyce jest znacznie szybszy, często bliski O(n / m) dzięki zastosowaniu heurystyk. Algorytm wykorzystuje dwie główne heurystyki:

1. Bad-character-heuristic – przeskakuje wzorzec tak, aby dopasować znak w tekście do ostatniego wystąpienia w wzorcu.
2. Good-suffix-heuristic – pozwala na przesunięcie wzorca na podstawie wcześniej dopasowanego sufiksu.

W naszym projekcie zastosowaliśmy wersję zgodną z wykładowym pseudokodem, z modyfikacją indeksowania od zera. Dzięki Boyerowi-Moore'owi możliwe jest błyskawiczne wyszukiwanie słów kluczowych w dużych plikach wynikowych, bez konieczności pełnego przeszukiwania każdego znaku tekstu.

Poprawność rozwiązania

Poprawność działania projektu opiera się na sprawdzonych i powszechnie stosowanych algorytmach komputerowych, których skuteczność i zgodność matematyczna zostały wielokrotnie potwierdzone w literaturze naukowej oraz w praktycznych zastosowaniach.

Algorytm Successive Shortest Path

Zaimplementowane rozwiązanie korzysta z algorytmu *Successive Shortest Path* (SSP), który iteracyjnie zwiększa przepływ w sieci rezydualnej wzdłuż najtańszych ścieżek od źródła do ujścia. Każda iteracja polega na znalezieniu takiej ścieżki i przesłaniu największego możliwego przepływu wzdłuż niej. Kluczowym założeniem algorytmu jest to, że przed i po każdej augmentacji utrzymywany jest brak cykli o ujemnym koszcie w grafie rezydualnym — co zapewnia, że przepływ jest optymalny kosztowo. Dowód tego faktu przeprowadzono indukcyjnie. Założono, że przed augmentacją graf rezydualny nie ma ujemnych cykli, a następnie pokazano, że augmentacja po najkrótszej ścieżce nie może ich wprowadzić (w przeciwnym razie znaleźlibyśmy krótszą ścieżkę, co jest sprzeczne z ideą minimalnego kosztu).

W momencie, gdy nie da się już zwiększyć przepływu lub osiągnięto już przepływ zamierzony, mamy przepływ maksymalny. Jeśli dodatkowo graf rezydualny nie zawiera cykli ujemnych, oznacza to, że uzyskany przepływ jest o minimalnym koszcie. To twierdzenie stanowi podstawę poprawności SSP i wynika z ogólnego warunku optymalności dla problemu minimalnego kosztu przepływu.

Ponieważ w grafie rezydualnym mogą pojawiać się krawędzie o ujemnym koszcie, do wyszukiwania najkrótszych ścieżek stosujemy potencjały wierzchołkowe, znane z algorytmu Johnsona. Dla każdego wierzchołka u przypisujemy potencjał π(u). Każdej krawędzi przypisujemy zredukowany koszt:

$$c_{\pi}(u,v) = c(u,v) + \pi(u) - \pi(v)$$

Gdzie c(u, v) to faktyczny koszt jednostkowy danej krawędzi. Teraz, gdy pierwotny graf nie zawiera ujemnych cykli, najkrótsze odległości od pewnego wierzchołka (np. od źródła s) istnieją i mogą służyć za wcześniej wspomniane potencjały. Wtedy dla każdej krawędzi (u, v) mamy:

$$dist(v) \leq dist(u) + c(u,v)$$

Co przekłada się na nieujemność zredukowanych kosztów:

$$c_\pi(u,v) = dist(u) + c(u,v) - dist(v) \geq 0$$

SSP rozpoczyna się z π(u) = 0, a po każdej iteracji potencjały π(u) każdego wierzchołka u są aktualizowane jako odległości od źródła do danego wierzchołka w grafie kosztów zredukowanych. W ten sposób możemy bezpiecznie stosować algorytm Dijkstry do wyznaczania najkrótszych ścieżek, mimo że oryginalne koszty krawędzi mogą być ujemne.

Cały algorytm gwarantuje poprawność rozwiązania poprzez trzy własności:

- Przepływ jest dopuszczalny względem pojemności krawędzi, przez które przepływ ten jest przesyłany,
- algorytm kończy się dopiero wtedy, gdy nie istnieje żadna ścieżka pozwalająca zwiększyć przepływ lub gdy już go osiągnięto, co implikuje przepływ maksymalny,
- brak cykli o ujemnym koszcie w grafie rezydualnym podczas działania algorytmu oraz na jego końcu oznacza, że koszt przepływu jest minimalny wśród wszystkich przepływów o tej samej wartości.

Źródło: courses.grainger.illinois.edu

Algorytm sortowania przez scalanie

Algorytm Merge Sort jest jednym z podstawowych algorytmów sortowania. Dla kompletności dokumentacji przedstawiamy skrótowe uzasadnienie poprawności:

Algorytm dzieli tablicę rekurencyjnie aż do jednoelementowych (lub pustych) podtablic, które są domyślnie posortowane. Następnie procedura merge prawidłowo scala dwie posortowane podtablice w jedną większą posortowaną tablicę. Zakładając, że podtablice są poprawnie posortowane (założenie indukcyjne), oraz że merge działa poprawnie, cała tablica po wykonaniu merge sort będzie posortowana.

Z tych powodów, przy zachowaniu poprawności rekurencji i operacji scalania, cały algorytm działa poprawnie.

Algorytm Grahama do wyznaczania otoczki wypukłej

Poprawność zastosowanego algorytmu została omówiona i formalnie udowodniona podczas wykładów, w związku z czym pomijamy jej szczegółowy dowód w niniejszej dokumentacji.

Algorytm winding number

Algorytm liczby owinięć (*winding number*) nie był omawiany na wykładzie, jednak opiera się na znanym i ugruntowanym podejściu geometrycznym. W skrócie:

Założenie: Dla punktu p obliczana jest suma kątów pomiędzy wektorami prowadzącymi od p do kolejnych wierzchołków wielokąta. Jeśli ta suma wynosi 0, oznacza to, że punkt znajduje się na zewnątrz wielokąta. Jeśli suma wynosi 2π (lub wielokrotność 2π), to punkt znajduje się wewnątrz. Dla wielokąta o wierzchołkach v1,v2,…,vn i badanego punktu p, obliczamy:

$$W(p) = \frac{1}{2\pi} \sum_{i=1}^{n} \angle(v_i - p, v_{i+1} - p)$$

gdzie ∠(a, b) to orientowany kąt pomiędzy wektorami a i b, a v n+1=v 1.

Interpretacja wyników jest następująca jeśli :

W(p)=0 → punkt na zewnątrz,

W(p)=±1 → punkt wewnątrz (dla prostych wielokątów),

|W(p)|>1 → punkt wewnątrz wielokąta samo przecinającego się (np. „ósemki”).

Poprawność wynika z faktu, że suma kątów obrotu wokół punktu p jest niezerowa tylko wtedy, gdy kontur faktycznie otacza punkt. Metoda jest odporna na lokalne zakrzywienia i działa niezależnie od wypukłości lub złożoności kształtu.

Rodzaj wykorzystanej technologii, języków programowania:

C++ – główna logika programu, algorytmy geometryczne i grafowe

Python + Flask – aplikacja webowa, integracja z programem C++

HTML/CSS/JS/Figma – szablony interfejsu użytkownika

g++ – kompilacja programu

Informacja dodatkowa:

Po więcej informacji odnośnie kompilacji i uruchomienia czyt. README.md, raporty testów znajdują się w pliku Raporty testow/.